Q198. What is connection pooling? A technique where a pool of reusable database connections is maintained and shared among multiple requests/users, avoiding the overhead of repeatedly opening and closing new connections.

Q199. What is database replication lag? The delay between when a change is made on a primary/master database and when that change is propagated and visible on a replica/secondary database.

Q200. What are some best practices for database performance tuning? Use appropriate indexing, avoid SELECT * in production code, normalize appropriately (but denormalize selectively where needed for read performance), analyze query execution plans, use connection pooling, partition large tables, and regularly monitor and update database statistics.

SECTION B: SQL — 50 PRACTICE QUERIES WITH SOLUTIONS

Sample Schema Used for All Practice Questions

```sql
Employee (emp_id, name, department_id, manager_id, salary, hire_date, gender)
Department (department_id, department_name, location)
Projects (project_id, project_name, department_id, budget)
EmployeeProjects (emp_id, project_id, hours_worked)
```

Level 1: Basic (Q1–Q15)

Q1. Fetch all employee records.

```sql
SELECT * FROM Employee;
```

Q2. Fetch names and salaries of all employees.

```sql
SELECT name, salary FROM Employee;
```

Q3. Fetch employees with salary greater than 50000.

```sql
SELECT * FROM Employee WHERE salary > 50000;
```

Q4. Fetch employees whose name starts with 'A'.

```sql
SELECT * FROM Employee WHERE name LIKE 'A%';
```

Q5. Fetch all distinct department IDs from the Employee table.

```sql
SELECT DISTINCT department_id FROM Employee;
```

Q6. Sort employees by salary in descending order.

```sql
SELECT * FROM Employee ORDER BY salary DESC;
```

Q7. Fetch the total number of employees.

```sql
SELECT COUNT(*) FROM Employee;
```

Q8. Fetch employees hired after January 1, 2022.

```sql
SELECT * FROM Employee WHERE hire_date > '2022-01-01';
```

Q9. Fetch employees with a NULL manager_id.

```sql
SELECT * FROM Employee WHERE manager_id IS NULL;
```

Q10. Fetch employees whose salary is between 40000 and 80000.

```sql
SELECT * FROM Employee WHERE salary BETWEEN 40000 AND 80000;
```

Q11. Fetch employees from department 10 or department 20.

```sql
```

```sql
SELECT * FROM Employee WHERE department_id IN (10, 20);
```

Q12. Count the number of male and female employees separately.

```sql
SELECT gender, COUNT(*) FROM Employee GROUP BY gender;
```

Q13. Fetch the average salary of all employees.

```sql
SELECT AVG(salary) FROM Employee;
```

Q14. Fetch the highest and lowest salary in the company.

```sql
SELECT MAX(salary) AS highest, MIN(salary) AS lowest FROM Employee;
```

Q15. Update the salary of an employee with emp_id = 101 to 60000.

```sql
UPDATE Employee SET salary = 60000 WHERE emp_id = 101;
```

Level 2: Intermediate (Q16–Q35)

Q16. Fetch employee names along with their department names.

```sql
SELECT e.name, d.department_name
FROM Employee e
JOIN Department d ON e.department_id = d.department_id;
```

Q17. Fetch department-wise average salary.

```sql
SELECT department_id, AVG(salary) AS avg_salary
FROM Employee
GROUP BY department_id;
```

Q18. Fetch departments having more than 5 employees.

```sql
SELECT department_id, COUNT(*) AS emp_count
FROM Employee
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Q19. Fetch the second highest salary from the Employee table.

```sql
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Q20. Fetch employees who earn more than the average salary of their department.

```sql
SELECT e.name, e.salary, e.department_id
FROM Employee e
WHERE e.salary > (
  SELECT AVG(salary) FROM Employee e2 WHERE e2.department_id = e.department_id
);
```

Q21. Fetch employee names along with their manager's name (self join).

```sql
SELECT e.name AS employee, m.name AS manager
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id;
```

Q22. Fetch employees who are not assigned to any project.

```sql
SELECT e.name
FROM Employee e
LEFT JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id
WHERE ep.project_id IS NULL;
```

Q23. Fetch the total budget of projects handled by each department.

```sql
SELECT department_id, SUM(budget) AS total_budget
FROM Projects
GROUP BY department_id;
```

Q24. Fetch duplicate employee names in the table.

```sql
SELECT name, COUNT(*)
FROM Employee
GROUP BY name
HAVING COUNT(*) > 1;
```

Q25. Fetch employees along with the number of projects they are working on.

```sql
SELECT e.name, COUNT(ep.project_id) AS project_count
FROM Employee e
LEFT JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id
GROUP BY e.name;
```

Q26. Fetch the top 3 highest paid employees.

```sql
SELECT * FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

Q27. Fetch employees who joined in the year 2023.

```sql
SELECT * FROM Employee
WHERE YEAR(hire_date) = 2023;
```

Q28. Fetch department names that have no employees.

```sql
```

```sql
SELECT d.department_name
FROM Department d
LEFT JOIN Employee e ON d.department_id = e.department_id
WHERE e.emp_id IS NULL;
```

Q29. Fetch the employee(s) with the highest salary in each department.

```sql
SELECT e.*
FROM Employee e
WHERE e.salary = (
  SELECT MAX(salary) FROM Employee e2 WHERE e2.department_id = e.department_id
);
```

Q30. Fetch a list of employees along with a label 'High', 'Medium', or 'Low' based on salary.

```sql
SELECT name, salary,
CASE
  WHEN salary >= 80000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_band
FROM Employee;
```

Q31. Fetch employees whose salary is above the overall company average, sorted descending.

```sql
SELECT * FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee)
ORDER BY salary DESC;
```

Q32. Fetch the total hours worked per project.

```sql
```

```sql
SELECT project_id, SUM(hours_worked) AS total_hours
FROM EmployeeProjects
GROUP BY project_id;
```

Q33. Fetch employee names containing the letter 'e' but not starting with 'e'.

```sql
SELECT name FROM Employee
WHERE name LIKE '%e%' AND name NOT LIKE 'e%';
```

Q34. Fetch the count of employees for every department, including departments with zero employees.

```sql
SELECT d.department_name, COUNT(e.emp_id) AS emp_count
FROM Department d
LEFT JOIN Employee e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

Q35. Delete employees who have not been assigned to any project.

```sql
DELETE FROM Employee
WHERE emp_id NOT IN (SELECT DISTINCT emp_id FROM EmployeeProjects);
```

Level 3: Advanced (Q36–Q50)

Q36. Fetch the Nth highest salary (parameterized) using a subquery.

```sql
SELECT DISTINCT salary
FROM Employee e1
WHERE (N - 1) = (
  SELECT COUNT(DISTINCT salary)
  FROM Employee e2
  WHERE e2.salary > e1.salary
);
```

Q37. Fetch the Nth highest salary using LIMIT/OFFSET.

```sql
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET N-1;
```

Q38. Fetch employees along with their department's rank based on salary (window function).

```sql
SELECT name, department_id, salary,
RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dept_rank
FROM Employee;
```

Q39. Fetch the running total of salaries ordered by hire_date.

```sql
SELECT name, hire_date, salary,
SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM Employee;
```

Q40. Fetch employees who earn more than their manager.

```sql
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_sal
FROM Employee e
JOIN Employee m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q41. Find departments where the total salary exceeds a specific project's budget.

```sql
```

```sql
SELECT dept_salary.department_id, dept_salary.total_salary
FROM (
  SELECT department_id, SUM(salary) AS total_salary
  FROM Employee GROUP BY department_id
) dept_salary
JOIN Projects p ON dept_salary.department_id = p.department_id
WHERE dept_salary.total_salary > p.budget;
```

Q42. Fetch the median salary of employees (approach for databases without MEDIAN()).

```sql
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS rn, COUNT(*) OVER () AS
  FROM Employee
) sub
WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2));
```

Q43. Fetch employees who work on all projects that department 10 works on.

```sql
SELECT ep.emp_id
FROM EmployeeProjects ep
WHERE ep.project_id IN (
  SELECT project_id FROM Projects WHERE department_id = 10
)
GROUP BY ep.emp_id
HAVING COUNT(DISTINCT ep.project_id) = (
  SELECT COUNT(*) FROM Projects WHERE department_id = 10
);
```

Q44. Find the difference between each employee's salary and the next highest-paid employee's salary (using LAG/LEAD).

```sql
SELECT name, salary,
salary - LEAD(salary) OVER (ORDER BY salary DESC) AS salary_gap
FROM Employee;
```

Q45. Fetch the top 2 highest paid employees in each department (window function).

```sql
SELECT * FROM (
  SELECT name, department_id, salary,
  DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk
  FROM Employee
) ranked
WHERE rnk <= 2;
```

Q46. Find employees hired in the same month as their manager.

```sql
SELECT e.name AS employee, m.name AS manager
FROM Employee e
JOIN Employee m ON e.manager_id = m.emp_id
WHERE MONTH(e.hire_date) = MONTH(m.hire_date);
```

Q47. Pivot department-wise employee counts into columns (conditional aggregation).

```sql
SELECT
  SUM(CASE WHEN department_id = 10 THEN 1 ELSE 0 END) AS dept_10,
  SUM(CASE WHEN department_id = 20 THEN 1 ELSE 0 END) AS dept_20,
  SUM(CASE WHEN department_id = 30 THEN 1 ELSE 0 END) AS dept_30
FROM Employee;
```

Q48. Find employees whose salary is greater than all employees in department 20 (ALL keyword).

```sql
SELECT name, salary
FROM Employee
WHERE salary > ALL (
  SELECT salary FROM Employee WHERE department_id = 20
);
```

Q49. Find the cumulative headcount of employees hired over time.

```sql
```

```sql
SELECT hire_date,
COUNT(*) OVER (ORDER BY hire_date) AS cumulative_headcount
FROM Employee;
```

Q50. Write a query using a CTE (Common Table Expression) to find department-wise top earner.

```sql
WITH RankedEmployees AS (
  SELECT name, department_id, salary,
  ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn
  FROM Employee
)
SELECT name, department_id, salary
FROM RankedEmployees
WHERE rn = 1;
```

End of document — 200 DBMS questions with detailed answers + 50 SQL practice queries with solutions.